

Midterm Essay

Course: Mining Massive Data Sets

1st Nguyen Khoi
Department of Computer Science
Ton Duc Thang University
522h0119@student.tdtu.edu.vn

2nd Huynh Le Quang
Department of Computer Science
Ton Duc Thang University
522h0105@student.tdtu.edu.vn

3rd Pham Huynh Tin
Department of Computer Science
Ton Duc Thang University
522h0150@student.tdtu.edu.vn

4th Doan Ngoc Bao
Department of Computer Science
Ton Duc Thang University
522h0099@student.tdtu.edu.vn

5th Nguyen Thanh An
Department of Computer Science
Ton Duc Thang University
tg_nguyenthanhan_cntt@tdtu.edu.vn

Abstract—This document is written to report on the midterm assignment for the Big Data course. It includes four tasks involving data manipulation on the “baskets.csv” file using methods through PySpark RDD and DataFrames, the development of the Apriori and PCY algorithms, and the visualization of the execution time for both algorithms on the provided dataset.

Index Terms—Big data, mining massive data sets

I. INTRODUCTION

II. TASK 1: PYSPARK RDDs & DATAFRAMES

In this section, we used PySpark RDDs and DataFrames to work with the provided dataset “baskets.csv”. For each operation, we saved the resulting output to a specified path and stored visualizations of the data.

A. Function f1: Select the top 100 most purchased items from the data set

To address the requirements using the given dataset, we designed a function f1 that takes the dataset’s path as input. Since each row in the dataset is comma-separated, we first split the columns accordingly and remove header. Next, we map all purchased items, which are located in the third column of the dataset, and reduce them to obtain the total count for each item. Finally, we sort the items in descending order and extract the top 100 most purchased items. The results are saved to a file, and a bar chart visualizing these top items is generated. The corresponding result is below:

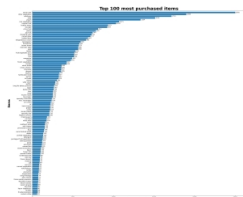


Fig. 1: Top 100 most purchased items

B. Function f2: Select the top 100 customers with the most shopping baskets

To address the requirements using the given dataset, we designed a function f2 that takes the dataset’s path as input. Since each row in the dataset is comma-separated, we first split the columns accordingly and remove the header. Next, we map all the pairs of values in the first and second columns to form pairs consisting of customer ID and purchase date. After that, we apply a reduction function using an anonymous function to calculate the total number of transactions per customer on a given day, while removing duplicate entries. Finally, we sort the data in descending order and extract the top 100 customers with the most purchases in a single day. The results are saved to a file, and a bar chart visualizing these top items is generated. The corresponding result is below:

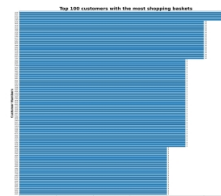


Fig. 2: Top 100 customers with most shopping baskets

C. Function f3: Count the number of times items have been purchased by month.

To address the requirements using the given dataset, we designed a function f3 that takes the dataset’s path and the target product as input. Since each row in the dataset is comma-separated, we first split the columns accordingly and remove the header. Next, we map all the pairs of values in the third and fifth columns to form pairs consisting of the target product and month. After that, we apply a reduction function using an anonymous function to calculate the total number of the number that

the target product were purchased in that month. Finally, we sort the data in descending order. The results are saved to a file, and a bar chart visualizing these top items is generated. The corresponding result is below:

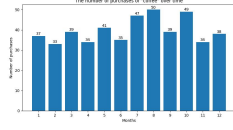


Fig. 3: Number of items got purchased by month

D. Function f4: Count the number of purchases made by customer each month.

To address the requirements using the given dataset, we designed a function f4 that takes the dataset's path and the target customer as input. Since each row in the dataset is comma-separated, we first split the columns accordingly and remove the header. Next, we map all the pairs of values in the first and fifth columns to form pairs consisting of the target customer and month. After that, we apply a reduction function using an anonymous function to calculate the total number of the number that the target customer shopped in that month. Finally, we sort the data in descending order. The results are saved to a file, and a bar chart visualizing these top items is generated. The corresponding result is below:

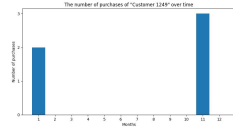


Fig. 4: Number of purchased made by customer

III. TASK 2: A-PRIORI ALGORITHM

In this section, the A-Priori algorithm was applied to the dataset 'basket.csv'. The dataset was processed to generate baskets, where each basket represents a collection of items purchased by a single customer on a specific date. Following the creation of these baskets, the results were written to a Hadoop directory by map reduce programs using java. These intermediate results were then leveraged to implement the A-Priori algorithm, structured into two classes, each representing one of the two passes of the algorithm. The implementation strategy is outlined as follows. [4].

A. Generating Baskets

To design the program for generating baskets from the dataset, a MapReduce model was implemented. The mapper class processes the dataset by splitting each line of data, discarding the header, and mapping it into key-value pairs structured as (Member-number + Date) - itemDescription.

After mapping the data into (customer-Date, itemDescription) pairs, we proceed to reduce them by using a set to ensure that only unique key-value pairs are retained,

ensuring that no duplicate items appear in the same basket. Additionally, we use a variable id to generate the final output in the format: id, basket.

In the final step, we pass the input and output path parameters on the HDFS system to execute the MapReduce program and observe the results from the file part-r-00000. The sample structure of result is below:

```
1, yogurt,sausage,whole milk,semi-finished bread
2, pastry,salty snack,whole milk
3, canned beer,misc. beverages
```

Fig. 5: Structure of Baskets

B. First Pass of A-Priori: List of all the frequent items in the data set

After extracting the baskets from the dataset, we implement a MapReduce program to perform the First Pass of the A-Priori algorithm.

Next the FirstPassMapper class processes each basket by splitting it into individual items and mapping them into key-value pairs in the form (item, 1), where each item in the basket is associated with a count of 1.

After the mapping step, the FirstPassReducer class reads the support threshold passed as a parameter. It sums the occurrences of each item across all baskets. If the total count of an item meets or exceeds the threshold, the item is identified as a frequent item. The output is written in the format id, frequent-item.

In the final step, to run the program, the input path, output path, and support threshold are provided as arguments. The results can be viewed in the part-r-00000 file within the HDFS system. The sample result structure is shown below:

```
1, Instant food products
2, UHT-milk
3, abrasive cleaner
```

Fig. 6: Sample result of the First Pass

C. Second Pass of A-Priori: List of all the frequent pairs in the data set

After obtaining the frequent items from the First Pass, we implement a MapReduce program to execute the Second Pass of the A-Priori algorithm. [1].

The SecondPassMapper class begins by reading the file containing frequent items, identified based on the support threshold in the First Pass. These frequent items are stored in a list for further processing.

In the mapping phase, each basket's items are processed by reading their values. For each basket, all items are stored in a list, and then nested loops are used to generate pairs of items. If both items in a pair are frequent (i.e., found in the list of frequent items), they are paired together and mapped as key-value pairs in the format item1,item2,1.

After mapping, the SecondPassReducer class processes the pairs by reducing them into the format pair-id, pair. Each pair of items is identified with a unique basket ID.

To run the program, the input path, output path, and frequent items path (generated from the First Pass) are passed as parameters. The final results are stored in the part-r-00000 file within the HDFS system. The sample result structure is shown below:

```
1,Instant food products,beef
2,Instant food products,beverages
3,Instant food products,bottled beer
```

Fig. 7: Sample structure of the Second Pass

IV. TASK 3: PCY ALGORITHM

In this section, we applied the PCY algorithm using PySpark DataFrame on the dataset basket.csv. We processed the data to create baskets representing items purchased by individual customers on specific dates. We then developed a PCY algorithm class based on an object-oriented model, allowing for flexible definitions of support threshold s and confidence threshold c to identify frequent item pairs and extract association rules.

A. Generating Basket from dataset

To retrieve all the baskets, we used PySpark DataFrame to read the data file from the specified path. We then grouped the two columns Member-number and Date, aggregating all the items into a list named items. Finally, we saved the items column to a CSV file named transactions.csv. The sample result structure is shown below:

```
,items
0,['specialty chocolate', 'other vegetables']
1,['beef', 'dessert']
2,['yogurt', 'beverages']
```

Fig. 8: Structure of Baskets

B. Implement the PCY class

a) Initialize and fit function: In the “init” method, the class assigns the values of s (support threshold) and c (confidence threshold) based on the parameters passed during initialization. Through the “fit” method, the class reads the transactions file mentioned above and converts the items column into an array of strings. Additionally, the class attributes are set to the values returned by the functions defined below.

b) Function get-items: get the items list and number of each item: This function is designed to return a dataframe of all items within the baskets, structured as “id, item, count”. It does this by exploding the “items” column, which contains lists of items, into individual items. Then, it groups these items, counts how many times each item appears, and assigns a unique item ID to each.

c) Function get-pairs: get all distinct pairs of from all the basket and number of each pair: This function is designed to return a dataframe by using a create-pairs function to creates all possible 2-item combinations from a list of items, then apply this function to items column for generating pairs for each transaction. Exploding the pairs column to break down each list of pairs into individual rows and grouping the pairs and counting their occurrences across all transactions.

d) Function get-buckets: processes pairs of items to create buckets and Hashing function: The ‘get-buckets()’ function processes the item pairs to create buckets for the PCY algorithm. It begins by extracting ‘item1’ and ‘item2’ from each pair and retains their counts. The function then joins these pairs with their corresponding item information to retrieve ‘item ids’ and counts for both items. Each pair is assigned to a bucket based on the hashing formula

$$item1id + item2id \mod 25 \quad (1)$$

Then grouping them into 25 buckets. It subsequently aggregates the counts of pairs within each bucket and returns a DataFrame that includes details about each pair, their counts, and their assigned bucket count.

e) Function get-bit-map: to creates a bitmap representation of item pairs based on their counts in the buckets: This function return a new DataFrame bit-map, which now includes a new column bit that contains these binary values (1s and 0s) corresponding to whether the item pairs are considered frequent based on the specified threshold. [1].

f) Function get-frequent-pairs: to get all the frequent pairs form the basket using the bit map DataFrame: The “get-frequent-pairs()” function identifies frequent pairs from the bit map generated in the PCY algorithm. It filters the “bit-map” DataFrame to retain only those entries where the bit value is equal to 1, indicating that the corresponding bucket met the support threshold during the bucketing process. Additionally, it applies further filters to ensure that the count of item1 and item2 in each pair is greater than or equal to the specified support threshold s . The resulting DataFrame contains only the pairs that are considered frequent based on these criteria. [1]. The sample result structure is shown below:

```
pair,pair count,item1 id,item1,item1 count,item2 id,item2,item2 count,bucket,bucket count,bit
["beverages", 'dessert'],4,1,beverages,248,99,dessert,353,0,1665,1
["beverages", 'butter milk'],6,1,beverages,248,124,butter milk,263,0,1665,1
["beverages", 'pastry'],7,1,beverages,248,149,pastry,774,0,1665,1
```

Fig. 9: Structure result of the Frequent Pairs

g) Function get-association-rules: generates association rules from the frequent pairs identified in the previous step: The function retrieves the frequent pairs DataFrame then creates two separate DataFrames df1 and df2. Where df1: Defines rules with item1 as the antecedent and item2 as the consequent, calculating the confidence as the ratio of the pair count to the count of item1. df2: Defines the reverse rules with item2 as the antecedent and item1 as

the consequent, calculating confidence similarly. Finally, it combines both DataFrames using a union operation and filters the results to retain only those rules with a confidence level greater than or equal to the specified threshold c . The result is a DataFrame of association rules along with their confidence values. [2]. The sample result structure is shown below:

```
antecedent,consequent,confidence
cat food,whole milk,0.14124293785310735
domestic eggs,whole milk,0.14234234234234233
processed cheese,rolls/buns,0.14473684210526316
```

Fig. 10: Structure result of the Association Rules

V. TASK 4: COMPARISON A-PRIORI ALGORITHM AND PCY ALGORITHM

The implementation of both the PCY and A-Priori algorithms using Pandas DataFrame remains consistent in terms of theoretical framework and algorithmic principles. [3].

To facilitate a comparison of execution time for both algorithms across datasets of increasing sizes, we developed a function that segments the dataset based on a specified percentage and outputs a separate CSV file. Subsequently, we employed a library to measure the runtime of both algorithms using different dataset files, maintaining the same threshold for each test. The chart comparing the execution times of the A-Priori and PCY algorithms with different size of dataset is illustrated as follows:

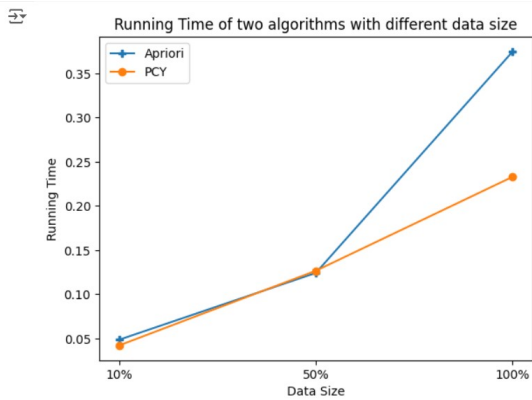


Fig. 11: Execution times comparison chart

The chart comparing the frequent pairs of the A-Priori and PCY algorithms with different threshold is illustrated as follows:

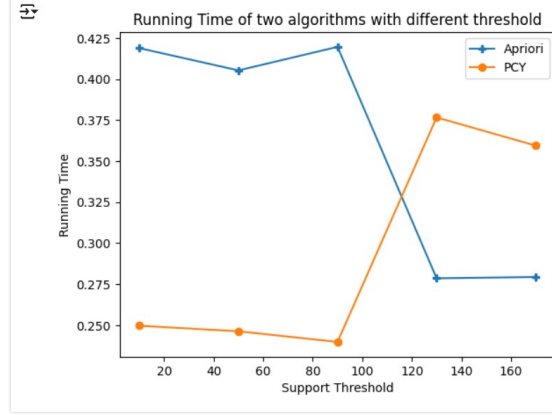


Fig. 12: Frequent pairs comparison chart

Our findings indicate that the execution time of the PCY algorithm generally outperforms that of A-Priori while also requiring less memory.

CONTRIBUTIONS

Contribution of each member:

- Nguyen Khoi: 100
- Huynh Le Quang: 100
- Pham Huynh Tin: 100
- Doan Ngoc Bao: 100

SELF-EVALUATION

Completion of each task:

- Task 1: 100
- Task 2: 100
- Task 3: 100
- Task 4: 100
- Task 5: 100

CONCLUSION

Summary:

- Task 1: Use PySpark and RDDs DataFrame to process the provided data and verify the results through visualization.
- Task 2: Implement the A-Priori algorithm using the MapReduce model in Java, run it via HDFS, and validate the results by checking the threshold and frequent item pairs.
- Task 3: Use PySpark DataFrames to implement the PCY algorithm to identify frequent pairs and association rules, and check whether the results meet the specified threshold and confidence requirements.
- Task 4: Split the data, measure execution time, and verify the results, visualizing them to monitor progress.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to MSc. Nguyen Thanh An for his thorough guidance and support, which enabled our group to successfully complete this report.

REFERENCES

- [1] "Association Rules and Frequent Itemsets,"[Online],Available: <http://infolab.stanford.edu/~ullman/mining/assocrules.pdf>?
- [2] Zhang, Q. H. (2017). Research on Association Rules Algorithm for Big Data. Ph.D. Thesis, Xi'an: Xi'an University of Science and Technology.
- [3] Zhou, J. Z., Liu, J., Yu, C. Y. (2010). Research and Application of Data Mining Based on Web Log. Science, Technology and Engineering, 10, 2762-2766.
- [4] Kamlesh Kumar Pandey, Diwakar Shukla. (2018). "Mining on Relationships in Big Data era using Improve Apriori Algorithm with MapReduce Approach". [Online] Available: <https://ieeexplore.ieee.org/abstract/document/8933674/authors>
- [5] Zhu, Y. (2014). Research on the Improvement of Apriori Algorithm Based on Array. Ph.D. Thesis, Harbin: Harbin normal University.